GoCare: dettaglio e rifiuto delle richieste

API associazione #2 e #3, più le correzioni alla #1 · branch **feature/APIAssociation** · commit 08fb6f4 · 21 ottobre 2026

Questo documento spiega che cosa fanno le nuove API dell'associazione e come è scritto il codice. Le due API completano il lato associazione del flusso *richiesta → accettazione → viaggio*: dopo aver visto la lista delle richieste in attesa (#1), l'associazione può aprirne una (#2) e accettarla (API già esistente) oppure rifiutarla (#3).

Stato: il codice è sul branch ma **non è ancora stato compilato né testato**, perché nell'ambiente in cui è stato scritto non era disponibile `dotnet`. Prima della pull request va eseguito `dotnet build` in locale.

In sintesi

#	Endpoint	Cosa fa
1	`GET /association/requests`	Già esistente. Aggiunto `.AsNoTracking()` e corretto il nome del file del controller.
2	`GET /association/requests/{id}`	Dettaglio di una richiesta in attesa per cui l'associazione è candidata.
3	`POST /association/requests/{id}/decline`	Rifiuto della richiesta, con motivo facoltativo. Se hanno rifiutato tutte le candidate, la richiesta diventa *non coperta* e il caregiver viene avvisato.

File toccati

File	Modifica
`Controllers/TransportRequest/ListPendingRequestsController.cs`	Rinominato (era ListPendindRequestsController.cs)
`Services/Transport/ListPendingRequestsService.cs`	Aggiunto .AsNoTracking()
`Controllers/TransportRequest/PendingRequestDetailController.cs`	Nuovo: API #2
`Services/Transport/PendingRequestDetailService.cs`	Nuovo: API #2
`Controllers/TransportRequest/DeclineTransportRequestController.cs`	Nuovo: API #3
`Dtos/Transport/Requests/DeclineTransportRequest.cs`	Nuovo: body della #3
`Services/Transport/DeclineTransportRequestService.cs`	Nuovo: API #3
`Services/Transport/CoverageEvaluator.cs`	Nuovo: regola di copertura usata dalla #3
`Program.cs`	Registrati i tre nuovi service

Tutti i percorsi sono relativi a `BackEnd/src/GoCare/`. Non serve nessuna migrazione: le tabelle usate esistevano già.

Il contesto: candidate e rifiuti

Quando un caregiver crea una richiesta di trasporto, `CreateTransportService` sceglie le associazioni che coprono la zona e le salva in **TransportRequestCandidates**. Questa lista viene congelata al momento della creazione: se un'associazione cambia in seguito i comuni coperti, le richieste già create non cambiano.

Ogni volta che un'associazione rinuncia a una richiesta si scrive una riga in **TransportRequestRejections**, con un `Kind`: `Declined` (rifiuto prima di accettare, quello di questa API) oppure `CancelledAfterAcceptance` (disdetta dopo aver accettato, API #6 non ancora fatta).

Le tre API dell'associazione usano lo stesso criterio per dire che una richiesta è "visibile":

- lo stato della richiesta è `Pending`;
- l'associazione è fra le candidate;
- l'associazione non l'ha già rifiutata.

L'id dell'associazione viene sempre letto dal token JWT (claim `sub`), mai dal client: un'associazione non può agire al posto di un'altra.

Correzioni alla #1: GET /association/requests

AsNoTracking. La query proietta gli indirizzi (`StartAddress`, `EndAddress`), che in EF Core sono *owned type*. Se si proiettano con il change tracker attivo, EF lancia un'eccezione e l'endpoint risponde 500. Con `.AsNoTracking()` EF legge i dati senza tracciarli, che per una lista in sola lettura è anche più veloce. È lo stesso fix già fatto in multipleFix sulle altre liste.

Nome del file. Il controller si chiamava `ListPendindRequestsController.cs` mentre la classe dentro era già `ListPendingRequestsController`. Il file è stato rinominato con `git mv`, quindi Git conserva la storia.

```
using GoCare.Data;
using GoCare.Dtos.Transport.Responses;
using GoCare.Models.Enums;

using Microsoft.EntityFrameworkCore;

namespace GoCare.Services.Transport;

// richieste in attesa che l'associazione può ancora accettare
public sealed class ListPendingRequestsService(GoCareDbContext db)
{
    public async Task<List<AssociationTransportResponse>> ListAsync(Guid associationId,
        CancellationToken ct)
    {
        return await db.TransportRequests.AsNoTracking()
            .Where(t => t.RequestStatus == ETripRequestStatus.Pending
                // sono fra le candidate di QUESTA richiesta
                && db.TransportRequestCandidates.Any(c => c.TransportRequestId == t.Id &&
        c.AssociationId == associationId)
                // e non l'ho già rifiutata o disdetta
                && !db.TransportRequestRejections.Any(r => r.TransportRequestId == t.Id &&
        r.AssociationId == associationId))
            .OrderBy(t => t.DepartureDateHour) // prima le più urgenti
            .Select(t => new AssociationTransportResponse(
                t.Id, t.TripType, t.TripDirection, t.DepartureDateHour, t.ReturnDateHour,
                t.StartAddress, t.EndAddress, t.RequestStatus))
            .ToListAsync(ct);
    }
}
```

API #2: GET /association/requests/{id}

Che cosa fa

Restituisce il dettaglio di una sola richiesta in attesa. Serve all'app dell'associazione quando l'operatore tocca una riga della lista e deve decidere se accettare o rifiutare.

Aspetto	Valore
Ruolo richiesto	Association (token JWT)
Parametro	`id`: Guid della richiesta di trasporto
Risposta 200	Un `AssociationTransportResponse`, lo stesso oggetto della lista #1
Risposta 404	La richiesta non esiste, non è più in attesa, l'associazione non è candidata oppure l'ha già rifiutata
Risposta 401 / 403	Token mancante, o token di un ruolo diverso da Association

Esempio di risposta


```
{
  "id": "3f2c9a1e-8b4d-4c6a-9e21-5d7f0b3a6c18",
  "tripType": "Visit",
  "tripDirection": "RoundTrip",
  "departureDateHour": "2026-10-28T08:30:00+00:00",
  "returnDateHour": "2026-10-28T12:00:00+00:00",
  "startAddress": { "street": "Via Roma", "number": "12", "postalCode": "90100",
                    "city": "Palermo", "province": "PA" },
  "endAddress":   { "street": "Via del Vespro", "number": "129", "postalCode": "90127",
                    "city": "Palermo", "province": "PA" },
  "requestStatus": "Pending"
}
```


Gli enum escono come stringhe perché in `Program.cs` è registrato `JsonStringEnumConverter`. I dati sono fittizi.

Perché 404 e non 403

Se la richiesta esiste ma non riguarda l'associazione, la risposta è comunque 404. Un 403 direbbe "questa richiesta esiste, ma non è tua", cioè rivelerebbe l'esistenza di richieste di altri. È la stessa scelta di `AcceptedTransportDetailService`.

Il codice

Il service riusa esattamente il filtro della lista, aggiungendo solo `t.Id == transportId`. Tutta la condizione diventa una sola query SQL: le due `Any(...)` sono tradotte in sottoquery `EXISTS`. `SingleOrDefaultAsync` restituisce `null` se non trova niente, e l'operatore `??` trasforma il null in `NotFoundException`, che il `GlobalExceptionHandler` converte in 404.

```
using GoCare.Data;
using GoCare.Dtos.Transport.Responses;
using GoCare.Errors;
using GoCare.Models.Enums;

using Microsoft.EntityFrameworkCore;

namespace GoCare.Services.Transport;

// dettaglio di una richiesta in attesa: stesso filtro di ListPendingRequestsService, su un solo id
public sealed class PendingRequestDetailService(GoCareDbContext db)
{
    public async Task<AssociationTransportResponse> GetAsync(Guid associationId, Guid transportId,
        CancellationToken ct)
    {
        return await db.TransportRequests.AsNoTracking()
            .Where(t => t.Id == transportId
                && t.RequestStatus == ETripRequestStatus.Pending
                && db.TransportRequestCandidates.Any(c => c.TransportRequestId == t.Id &&
        c.AssociationId == associationId)
                && !db.TransportRequestRejections.Any(r => r.TransportRequestId == t.Id &&
        r.AssociationId == associationId))
            .Select(t => new AssociationTransportResponse(
                t.Id, t.TripType, t.TripDirection, t.DepartureDateHour, t.ReturnDateHour,
                t.StartAddress, t.EndAddress, t.RequestStatus))
            .SingleOrDefaultAsync(ct)
            // 404 anche se esiste ma non è per me: non rivelo richieste altrui
            ?? throw new NotFoundException("Richiesta non trovata o non più disponibile per la tua
        associazione.");
    }
}
```

Il controller fa solo tre cose: legge l'id dal token, chiama il service e risponde 200.

```
using System.Security.Claims;

using GoCare.Models.Enums;
using GoCare.Services.Transport;

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace GoCare.Controllers.TransportRequest;

[ApiController]
[Route("association/requests")]
[Tags("Trasporti")]
[Authorize(Roles = nameof(EAccountRole.Association))]
public sealed class PendingRequestDetailController(PendingRequestDetailService service) :
        ControllerBase
{
    [HttpGet("{id:guid}")]
    public async Task<IActionResult> GetDetail(Guid id, CancellationToken ct)
    {
        var associationId = Guid.Parse(User.FindFirstValue("sub")!);

        var request = await service.GetAsync(associationId, id, ct);

        return Ok(request);
    }
}
```

API #3: POST /association/requests/{id}/decline

Che cosa fa

L'associazione dichiara che non prende in carico la richiesta. Da quel momento la richiesta sparisce dalla sua lista e dal suo dettaglio, ma resta disponibile per le altre candidate. Se l'ultima candidata rimasta rifiuta, nessuno può più coprire il viaggio: la richiesta passa allo stato `NotCovered` e il caregiver riceve una notifica, così può organizzarsi in altro modo.

Aspetto	Valore
Ruolo richiesto	Association (token JWT)
Parametro	`id`: Guid della richiesta di trasporto
Body	Facoltativo. `{ "reason": "Nessun mezzo disponibile" }`. Si può anche non mandare niente.
Risposta 204	Rifiuto registrato
Risposta 422	Il motivo supera 300 caratteri
Risposta 404	La richiesta non esiste
Risposta 403	L'associazione non è fra le candidate
Risposta 409	La richiesta non è più in attesa (accettata, annullata, non coperta), oppure l'associazione l'ha già rifiutata

I passi, in ordine

Passo	Che cosa succede
1. Motivo	Spazi tolti; un motivo vuoto diventa null. Se supera 300 caratteri: 422. Il limite è lo stesso della colonna Reason nel database.
2. Richiesta	Caricata dal database (con tracking, perché potrebbe cambiare stato). Se non c'è: 404.
3. Candidata?	Se l'associazione non è fra le candidate: 403. È lo stesso controllo di accept.
4. In attesa?	Se lo stato non è Pending: 409.
5. Già rifiutata?	Se esiste già un rifiuto di questa associazione: 409. Evita righe doppie.
6. Rifiuto	Nuova TransportRequestRejection con Kind = Declined, aggiunta al DbContext ma non ancora salvata.
7. Copertura	CoverageEvaluator controlla se ora tutte le candidate hanno rifiutato. Se sì: transport.RequestNotCovered(now).
8. Salvataggio	Un solo SaveChangesAsync: rifiuto e cambio di stato finiscono nella stessa transazione.
9. Notifica	Solo se la richiesta è diventata NotCovered: notifica RequestNotCovered al caregiver, push ed email.

Concorrenza

Cosa succede se un'associazione rifiuta mentre un'altra accetta nello stesso istante? La tabella delle richieste usa la colonna `xmin` di PostgreSQL come token di concorrenza. Se la richiesta è stata modificata dopo che l'abbiamo letta, `SaveChangesAsync` lancia `DbUpdateConcurrencyException`, che il service converte in 409. È lo stesso meccanismo "vince la prima" dell'API accept.

Il codice del service

```
using GoCare.Data;
using GoCare.Errors;
using GoCare.Models.Domain;
using GoCare.Models.Enums;

using Microsoft.EntityFrameworkCore;

namespace GoCare.Services.Transport;

public sealed class DeclineTransportRequestService(
    GoCareDbContext db, CoverageEvaluator coverage, NotificationDispatcher notifications)
{
    private const int ReasonMaxLength = 300; // come la colonna Reason di TransportRequestRejection

    public async Task DeclineAsync(Guid transportId, Guid associationId, string? reason,
        CancellationToken ct)
    {
        reason = string.IsNullOrWhiteSpace(reason) ? null : reason.Trim();
        if (reason is { Length: > ReasonMaxLength })
            throw new ValidationException(new Dictionary<string, string[]>
            {
                ["reason"] = [$"Il motivo non può superare {ReasonMaxLength} caratteri."]
            });

        var transport = await db.TransportRequests.SingleOrDefaultAsync(t => t.Id == transportId,
        ct)
            ?? throw new NotFoundException("Richiesta di trasporto non trovata.");

        var isCandidate = await db.TransportRequestCandidates.AnyAsync(
            c => c.TransportRequestId == transportId && c.AssociationId == associationId, ct);
        if (!isCandidate)
            throw new ForbiddenException("La tua associazione non è candidata per questa
        richiesta.");

        if (transport.RequestStatus is not ETripRequestStatus.Pending)
            throw new ConflictException("La richiesta non è più in attesa: non può essere
        rifiutata.");

        var alreadyDeclined = await db.TransportRequestRejections.AnyAsync(
            r => r.TransportRequestId == transportId && r.AssociationId == associationId, ct);
        if (alreadyDeclined)
            throw new ConflictException("La tua associazione ha già rifiutato questa richiesta.");

        var now = DateTimeOffset.UtcNow;
        db.TransportRequestRejections.Add(new TransportRequestRejection(
            Guid.NewGuid(), transportId, associationId, ERejectionKind.Declined, reason, now));

        var notCovered = await coverage.IsNotCoveredAsync(transport, ct);
        if (notCovered)
            transport.RequestNotCovered(now);

        try
        {
            await db.SaveChangesAsync(ct);
        }
        catch (DbUpdateConcurrencyException)
        {
            // un'altra associazione ha accettato nello stesso istante (xmin)
            throw new ConflictException("La richiesta è stata modificata nel frattempo: non è più
        in attesa.");
```

```
        }

        if (notCovered)
            await notifications.DispatchAsync(
                ENotificationSubject.Person,
                transport.RequestedById,
                ENotificationType.RequestNotCovered,
                "Richiesta di trasporto non coperta",
                "Nessuna associazione disponibile ha potuto prendere in carico la tua richiesta di
        trasporto.",
                ENotificationChannel.Push | ENotificationChannel.Email,
                transport.Id,
                ct);
    }
}
```

Il controller e il body facoltativo

Con `[ApiController]`, ASP.NET Core risponde 400 se un body atteso manca. L'attributo `[FromBody(EmptyBodyBehavior = EmptyBodyBehavior.Allow)]` permette invece una POST senza body: in quel caso `request` è null e `request?.Reason` passa null al service.

```
using System.Security.Claims;

using GoCare.Dtos.Transport.Requests;
using GoCare.Models.Enums;
using GoCare.Services.Transport;

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.ModelBinding;

namespace GoCare.Controllers.TransportRequest;

[ApiController]
[Route("association/requests")]
[Tags("Trasporti")]
[Authorize(Roles = nameof(EAccountRole.Association))]
public sealed class DeclineTransportRequestController(DeclineTransportRequestService service) :
        ControllerBase
{
    // il body è facoltativo: senza body il rifiuto è senza motivo
    [HttpPost("{id:guid}/decline")]
    public async Task<IActionResult> Decline(
        Guid id,
        [FromBody(EmptyBodyBehavior = EmptyBodyBehavior.Allow)] DeclineTransportRequest? request,
        CancellationToken ct)
    {
        var associationId = Guid.Parse(User.FindFirstValue("sub")!);

        await service.DeclineAsync(id, associationId, request?.Reason, ct);

        return NoContent();
    }
}
```

```
namespace GoCare.Dtos.Transport.Requests;

public sealed record DeclineTransportRequest(string? Reason);
```

CoverageEvaluator

Decide quando una richiesta non può più essere coperta. Il recap la prevede come classe a parte perché servirà anche ad altre parti: alla disdetta dopo l'accettazione (#6), all'eliminazione di un'associazione (#18) e al job che chiude le richieste scadute (soglia temporale PA-02). Per ora contiene una sola regola: **tutte le candidate hanno rifiutato**.

Come funziona:

- se la richiesta non è `Pending`, risponde subito false;
- legge dal database le associazioni candidate e quelle che hanno già un rifiuto salvato;
- aggiunge i rifiuti che sono nel `ChangeTracker` in stato *Added*, cioè aggiunti ma non ancora salvati. Senza questo passo non vedrebbe il rifiuto appena creato dalla #3, perché il salvataggio avviene dopo;
- risponde true se ogni candidata compare fra le associazioni che hanno rifiutato.

```
using GoCare.Data;
using GoCare.Models.Domain;
using GoCare.Models.Enums;

using Microsoft.EntityFrameworkCore;

namespace GoCare.Services.Transport;

// decide quando una richiesta in attesa diventa NotCovered.
// Per ora una sola regola: tutte le candidate hanno rifiutato. La soglia temporale (PA-02) andrà
        qui.
public sealed class CoverageEvaluator(GoCareDbContext db)
{
    public async Task<bool> IsNotCoveredAsync(TransportRequest transport, CancellationToken ct)
    {
        if (transport.RequestStatus is not ETripRequestStatus.Pending)
            return false;

        // candidate che non hanno ancora un rifiuto registrato (né salvato né in attesa di
        SaveChanges)
        var candidates = await db.TransportRequestCandidates
            .Where(c => c.TransportRequestId == transport.Id)
            .Select(c => c.AssociationId)
            .ToListAsync(ct);

        var rejectedSaved = await db.TransportRequestRejections
            .Where(r => r.TransportRequestId == transport.Id)
            .Select(r => r.AssociationId)
            .ToListAsync(ct);

        var rejectedPending = db.ChangeTracker.Entries<TransportRequestRejection>()
            .Where(e => e.State == EntityState.Added && e.Entity.TransportRequestId == transport.Id)
            .Select(e => e.Entity.AssociationId);

        var rejected = rejectedSaved.Concat(rejectedPending).ToHashSet();

        return candidates.All(rejected.Contains);
    }
}
```

Registrazione dei service

In `Program.cs` i tre nuovi service sono registrati come `Scoped`, come tutti gli altri: un'istanza per ogni richiesta HTTP, che condivide lo stesso `GoCareDbContext`. Questo è importante per `CoverageEvaluator`, che deve vedere il ChangeTracker dello stesso DbContext usato dal service del rifiuto.

```
builder.Services.AddScoped<ListPendingRequestsService>();
builder.Services.AddScoped<PendingRequestDetailService>();
builder.Services.AddScoped<CoverageEvaluator>();
builder.Services.AddScoped<DeclineTransportRequestService>();
```

Limiti noti

- **Due rifiuti nello stesso istante.** Se le ultime due candidate rifiutano insieme, ognuna vede solo il proprio rifiuto e non quello dell'altra. Entrambi i rifiuti vengono salvati, ma la richiesta resta `Pending` invece di diventare `NotCovered`. Il `CoverageTimeoutJob` previsto nel recap la recupererebbe, ma non esiste ancora.
- **Dati dell'assistito.** Il dettaglio restituisce gli stessi campi della lista. Se all'associazione servono altri dati minimi per decidere (PA-04), vanno aggiunti a un DTO dedicato.
- **Non compilato.** Il codice non è passato da `dotnet build` e non ci sono test automatici per le nuove API.

Come provarla

- Scaricare il branch: `git fetch origin feature/APIAssociation` e `git checkout feature/APIAssociation`.
- In `BackEnd/`: `dotnet build` e `dotnet test`.
- Da Swagger, con il token di un'associazione candidata: `GET /association/requests` per prendere un id, poi `GET /association/requests/{id}`.
- `POST /association/requests/{id}/decline` deve rispondere 204. Subito dopo, la lista non deve più contenere la richiesta e il dettaglio deve rispondere 404.
- Un secondo decline sulla stessa richiesta deve rispondere 409.
- Con una richiesta che ha una sola candidata: dopo il decline la richiesta deve risultare `NotCovered` e il caregiver deve ricevere la notifica. Con l'email in console, il testo compare nel log del backend.